Library Management System

Project Report

Internship Task 2

A Python-based Command-Line Application for Managing Books, Members, and Book Issue/Return Records

Prepared By	Thanishaa B
Language	Python 3
Interface	Command-Line (CLI)
Data Storage	JSON files (local, file-based)
Testing	Pytest — 15 automated unit tests
Repository Type	Public GitHub Repository
Date	August 2026

1. Introduction

Libraries — whether in schools, colleges, or public institutions — need a reliable way to keep track of their book collection, their registered members, and the flow of books being borrowed and returned. Doing this manually with registers is slow and error-prone: books go missing without a record, overdue fines are forgotten, and finding a specific title in a large catalogue takes far too long.

This project implements a **Library Management System** as a Python command-line application. It digitizes the core day-to-day operations of a small-to-medium library: cataloguing books, registering members, issuing and returning books, calculating overdue fines automatically, and letting staff search and filter records quickly.

1.1 Objectives

- Provide a simple, menu-driven interface for library staff to manage books, members, and loans without needing to know Python.
- Enforce real library business rules automatically (copy availability, per-member borrowing limits, overdue fines) instead of relying on manual bookkeeping.
- Persist all data reliably between sessions using file-based JSON storage, so no separate database server needs to be installed or configured.
- Handle invalid input and rule violations gracefully via a structured exception hierarchy, so the application never crashes on bad input.
- Keep the codebase modular and testable, with an automated test suite covering the core logic.

2. System Overview

The system is architected in four layers, each with a single, clear responsibility. This separation of concerns means the persistence layer can be swapped (e.g. for a real database) without touching the business logic, and the business logic can be reused behind a different interface (e.g. a web app) without modification.

Layer	File	Responsibility
Presentation	main.py	CLI menus, user input/output, top-level error display
Business Logic	library.py	Validation, business rules, orchestrating operations
Data Models	models.py	Book, Member, IssueRecord classes and their (de)serialization
Persistence	storage.py	Reading/writing JSON files safely and atomically
Error Handling	exceptions.py	Custom exception types for every failure category

3. Key Features

3.1 Book Management

Staff can add new books (title, author, ISBN, category, number of copies), view the full catalogue, update book details, and delete books. Each book tracks both its total copy count and how many copies are currently available, so the system always knows whether a title can be issued right now. Duplicate ISBNs are rejected, and a book cannot be deleted while any of its copies are still out on loan.

3.2 Member Management

Members are registered with a name, email address, and phone number. Email addresses are validated with a regular expression and must be unique across all members. A member with books currently checked out cannot be deleted, protecting the integrity of the loan records.

3.3 Issue & Return Books

Issuing a book creates a loan record with an issue date and a due date 14 days later, and decrements the book's available copy count. The system enforces a maximum of 3 books issued per member at any one time, and refuses to issue a book with zero copies available. Returning a book records the return date, restores the available copy count, and automatically computes a fine of ■5 for every day the book was returned late.

3.4 Search & Filter

Books can be searched by a free-text keyword matched against title, author, or ISBN, and can additionally be filtered by category or by availability (only show books with at least one free copy). Members can similarly be searched by name, email, or phone number.

3.5 Data Storage

All data — books, members, and issue records — is stored locally in human-readable JSON files inside the `data/` folder. Writes are performed atomically (written to a temporary file and then renamed into place) so an interrupted write can never leave a corrupted data file behind. Auto-incrementing IDs for books, members, and issue records are tracked in a small counters file to guarantee uniqueness across the application's lifetime.

3.6 Exception Handling

A dedicated hierarchy of exception classes (all inheriting from a common `LibraryError`) represents every category of failure the system can encounter: not-found errors, duplicate entries, business-rule violations (no copies available, membership limit reached), invalid input data, and storage failures. The CLI layer catches these exceptions and prints a friendly, specific message instead of letting a raw traceback reach the user, so the application never crashes during normal use.

Exception	Raised When
`BookNotFoundError`	A lookup uses a book ID that doesn't exist
`MemberNotFoundError`	A lookup uses a member ID that doesn't exist
`DuplicateBookError`	A new book reuses an existing ISBN
`DuplicateMemberError`	A new member reuses an existing email

`NoCopiesAvailableError`	Issuing a book with 0 copies available
`MembershipLimitError`	A member already has 3 books issued
`BookNotIssuedError`	Returning a book not on loan to that member
`InvalidDataError`	Empty/malformed input fails validation
`StorageError`	A JSON data file is missing, corrupt, or unwritable

4. Data Model

The system revolves around three core entities, shown below with their key fields.

Book	Member	IssueRecord
book_id (auto)	member_id (auto)	record_id (auto)
title	name	book_id (FK)
author	email (unique)	member_id (FK)
isbn (unique)	phone	issue_date
category	membership_date	due_date
total_copies		return_date
available_copies		fine

Each model class exposes `to_dict()` and `from_dict()` methods, which are the only points where a model knows about serialization — keeping the persistence format decoupled from the business logic.

5. Technology Stack

Component	Choice	Reason
Language	Python 3.8+	Simple, readable, widely taught — ideal for an internship project
Interface	Command-line (stdlib)	No extra dependencies; runs anywhere Python runs
Storage	JSON files	Human-readable, no database server to install/configure
Testing	pytest / unittest	Fast, standard tooling for automated regression testing
Version Control	Git + GitHub	Industry-standard for source control and collaboration

6. Testing

An automated test suite (`tests/test_library.py`) exercises the core business logic using Python's `unittest` framework, run via pytest. Each test runs against an isolated temporary data directory so it never touches real application data. The suite covers 15 scenarios across all major features:

- Adding books and members successfully, including default values.
- Rejecting duplicate ISBNs and duplicate member emails.
- Rejecting invalid input such as empty titles or malformed email addresses.
- Looking up a non-existent book or member correctly raises a not-found error.
- Issuing and returning a book updates available copy counts correctly.
- Issuing a book with zero copies available is blocked.

- Returning a book never issued to that member is blocked.
- The 3-book-per-member borrowing limit is enforced.
- Keyword search and availability filtering return the correct subset of books.

All 15 tests pass. The suite can be re-run at any time with:

```
python -m pytest tests/ -v
```

7. Sample Usage

Below is a real console session showing a book being issued to a member, taken directly from running the application:

```
Choose an option: 3
1. Issue a Book (max 3 per member)
2. Return a Book
Choose an option: 1
Book ID to issue: 1
Member ID: 1

✔ Book issued successfully. Due back by 2026-08-15.
[1] Book:1 Member:1 Issued:2026-08-01 Due:2026-08-15 | Not returned
```

8. Challenges & Learnings

- **Designing a clean exception hierarchy:** rather than reusing generic exceptions everywhere, creating specific exception classes made the CLI's error messages far more precise and the code more self-documenting.
- **Atomic file writes:** writing directly to the target JSON file risked leaving corrupted data if the program was interrupted mid-write. Writing to a temporary file and renaming it into place solved this safely.
- **Separating concerns:** keeping storage, business logic, and the CLI in separate modules made the code much easier to unit-test, since the tests could exercise `library.py` directly without any user input at all.

9. Conclusion

This project delivers a complete, working Library Management System that covers all required features: book management, member management, issuing and returning books with automatic fine calculation, keyword/category/availability search, durable JSON-based data storage, and thorough exception handling. Its layered architecture keeps the code easy to read, test, and extend — whether that means adding a graphical or web interface later, or migrating storage to a full database as the library's collection grows.

Report prepared by: **Thanishaa B**

— End of Report —